# Your first OpenFX plugin in DaVinci Resolve

A from-scratch guide for Windows: C++, CMake, Visual Studio Build Tools and the OpenFX SDK. It follows the route that actually worked, including every wall that cost days, so you can skip them.

**How to read this guide.** Everything marked **Tested** was run on Windows 11 with DaVinci Resolve 21 and a plugin that now loads, renders and has a working slider. Names and function signatures were checked against the OpenFX SDK headers on that machine, not written from memory. Anything marked **Resolve-specific** is how Resolve behaved, not something the OpenFX standard promises. Anything marked **Not tested** is a gap you should check yourself.

## Contents


- 1. The big picture: host, plugin, bundle
- 2. What you need installed
- 3. Folder layout
- 4. The smallest plugin that works
- 5. CMakeLists.txt
- 6. Build, stage and install
- 7. Making the plugin appear in Resolve
- 8. Troubleshooting: symptom, cause, fix
- 9. The debug log trick
- 10. Verify names in your own headers
- 11. Where to go next

# 1. The big picture

An OpenFX plugin is a normal Windows DLL that has been renamed to **.ofx**. DaVinci Resolve is the **host**. It never links to your plugin at build time. At runtime it scans a fixed folder, loads each `.ofx` file, and looks up two functions by name. From then on the host talks to your plugin by calling one single function with an action name, such as Describe or Render.

Think of it as a restaurant. The host is the customer, your plugin is the kitchen, and the actions are orders. The kitchen never walks out to the dining room. It only answers the orders it is handed, and it asks the host for ingredients (images, parameter values) through tables of function pointers called **suites**.

## The pipeline you are learning

```
Host (Resolve)
  -> loads your .ofx, calls OfxGetNumberOfPlugins / OfxGetPlugin
  -> gives you its function tables (suites) in the Load action
  -> Describe: what the plugin is
  -> DescribeInContext: its clips (and later its parameters)
  -> CreateInstance: one effect applied to one clip
  -> Render: produce one output frame from the source frame
```

## What each side is responsible for

| Resolve (host) | Your plugin |
|---|---|
| Finds and loads the .ofx, owns the UI, stores parameter values, owns keyframes, owns image memory | Describes itself, declares clips and parameters, reads the pixels it is given, writes the output pixels, releases everything it fetched |

## The Windows bundle

On Windows the host expects a **bundle**: a folder whose name ends in `.ofx.bundle`, with your DLL inside a fixed sub-path. This is the structure that loaded in Resolve 21 (**Tested**):

```
MyFirstPlugin.ofx.bundle\
  Contents\
    Win64\
      MyFirstPlugin.ofx          <- your renamed DLL
```

The bundle goes in the standard OFX plugin folder:

```
C:\Program Files\Common Files\OFX\Plugins\MyFirstPlugin.ofx.bundle
```

Writing to Program Files needs administrator rights. That is why the build step below must run as administrator.

## 2. What you need installed

| Tool | Used for | State on the tested machine |
|---|---|---|
| DaVinci Resolve | The host that loads and runs the plugin | Resolve 21 |
| Visual Studio Build Tools | The MSVC compiler and linker | Visual Studio 2026 Build Tools, x64 |
| CMake | Generates the build from CMakeLists.txt | CMake 4.4.3 |
| OpenFX SDK | The headers (ofxCore.h, ofxImageEffect.h, ofxParam.h ...) | Folder named openfx |
| VS Code (optional) | Editor and one-key build task | Used for Ctrl+Shift+B |

## 3. Folder layout

Keep the SDK and your plugin side by side. The CMake file below finds the SDK one level up:

```
D:\DavinciCoding\OpenfxProject\
  openfx\                    <- the OpenFX SDK (contains include\)
  MyFirstPlugin\
    CMakeLists.txt
    src\MyFirstPlugin.cpp
    build-install.bat        <- build + stage + install (section 6)
    bundle\                 <- staged bundle, created by the bat file
    log.txt                  <- written by the plugin (section 9)
```

Your own paths can differ. Only the relative link between the CMake file and the SDK matters, and it is controlled by one line (`OFX_SDK_DIR`).

## 4. The smallest plugin that works

A plugin needs exactly these pieces. Each one is something Resolve checks.

- **Two exported functions** with plain, undecorated names: `OfxGetNumberOfPlugins` and `OfxGetPlugin`. Resolve finds them by name.
- **An OfxPlugin struct**: API name, API version, a unique identifier string, version numbers, a `setHost` function and a `mainEntry` function.
- **setHost**: stores the host pointer and does nothing else. The header forbids calling OFX functions here.
- **mainEntry**: one function that receives every action as a string and returns a status.
- **Load** action: fetch the suites you need from the host.
- **Describe** and **DescribeInContext**: say what you are and declare your clips.
- **CreateInstance** and **DestroyInstance**: return OK (see section 8, this one cost the most time).
- **Render**: do the actual pixel work.

### The skeleton (the parts that load the plugin)

```
#include "ofxImageEffect.h"
#include <cstring>

static OfxHost *gHost = nullptr;
static OfxImageEffectSuiteV1 *gEffectHost = nullptr;
static OfxPropertySuiteV1    *gPropHost   = nullptr;
```

```
static void setHostFunc(OfxHost *h) { gHost = h; }      // store only

static OfxStatus onLoad(void)
{
    if (!gHost) return kOfxStatErrMissingHostFeature;
    gEffectHost = (OfxImageEffectSuiteV1 *)
        gHost->fetchSuite(gHost->host, kOfxImageEffectSuite, 1);
    gPropHost = (OfxPropertySuiteV1 *)
        gHost->fetchSuite(gHost->host, kOfxPropertySuite, 1);
    if (!gEffectHost || !gPropHost) return kOfxStatErrMissingHostFeature;
    return kOfxStatOK;
}

static OfxStatus describe(OfxImageEffectHandle effect)
{
    OfxPropertySetHandle props;
    gEffectHost->getPropertySet(effect, &props);
    gPropHost->propSetString(props, kOfxPropLabel, 0, "My First Plugin");
    gPropHost->propSetString(props, kOfxImageEffectPluginPropGrouping, 0, "OFX Learning");
    gPropHost->propSetString(props, kOfxImageEffectPropSupportedContexts, 0,
                             kOfxImageEffectContextFilter);
    gPropHost->propSetString(props, kOfxImageEffectPropSupportedPixelDepths, 0, kOfxBitDepthFloat);
    gPropHost->propSetString(props, kOfxImageEffectPropSupportedPixelDepths, 1, kOfxBitDepthShort);
    gPropHost->propSetString(props, kOfxImageEffectPropSupportedPixelDepths, 2, kOfxBitDepthByte);
    gPropHost->propSetInt(props, kOfxImageEffectPropSupportsMultipleClipDepths, 0, 0);
    return kOfxStatOK;
}

static OfxStatus describeInContext(OfxImageEffectHandle effect, OfxPropertySetHandle)
{
    OfxPropertySetHandle clipProps;
    gEffectHost->clipDefine(effect, kOfxImageEffectOutputClipName, &clipProps);
    gPropHost->propSetString(clipProps, kOfxImageEffectPropSupportedComponents, 0,
                             kOfxImageComponentRGBA);
    gEffectHost->clipDefine(effect, kOfxImageEffectSimpleSourceClipName, &clipProps);
    gPropHost->propSetString(clipProps, kOfxImageEffectPropSupportedComponents, 0,
                             kOfxImageComponentRGBA);
    return kOfxStatOK;
}

static OfxStatus pluginMain(const char *action, const void *handle,
                            OfxPropertySetHandle inArgs, OfxPropertySetHandle outArgs)
{
    try {
        OfxImageEffectHandle effect = (OfxImageEffectHandle) handle;
        if (strcmp(action, kOfxActionLoad) == 0)           return onLoad();
        if (strcmp(action, kOfxActionDescribe) == 0)       return describe(effect);
        if (strcmp(action, kOfxImageEffectActionDescribeInContext) == 0)
                                                           return describeInContext(effect, inArgs);
        if (strcmp(action, kOfxActionCreateInstance) == 0)  return kOfxStatOK;
        if (strcmp(action, kOfxActionDestroyInstance) == 0) return kOfxStatOK;
        if (strcmp(action, kOfxImageEffectActionRender) == 0) return kOfxStatOK; // first: do nothing
    }
    catch (...) { return kOfxStatErrUnknown; }   // never let an exception reach the host
    return kOfxStatReplyDefault;                  // actions we do not handle
}

static OfxPlugin gPlugin = {
    kOfxImageEffectPluginApi, kOfxImageEffectPluginApiVersion,
    "com.yourname.MyFirstPlugin",   // unique identifier, no spaces
    1, 0, setHostFunc, pluginMain };

OfxExport OfxPlugin *OfxGetPlugin(int nth) { return nth == 0 ? &gPlugin : nullptr; }
OfxExport int OfxGetNumberOfPlugins(void)  { return 1; }
```

Note the Render line above returns OK without drawing anything, so the clip would be unchanged. A real Render has to fetch the source and output images and write to the output. The next section of the course (passthrough, then invert) covers it. A render function that works is in the course file MyFirstPlugin.cpp.

# 5. CMakeLists.txt

An OFX plugin is a DLL the host loads at runtime, so it is a CMake **MODULE** library, renamed to .ofx. This file produced a loading plugin (**Tested**):

```
cmake_minimum_required(VERSION 3.15)
project(MyFirstPlugin LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Where the OpenFX SDK lives. Change this one line if the SDK moves.
set(OFX_SDK_DIR "${CMAKE_CURRENT_SOURCE_DIR}/../openfx" CACHE PATH "Path to the OpenFX SDK")

add_library(MyFirstPlugin MODULE src/MyFirstPlugin.cpp)
target_include_directories(MyFirstPlugin PRIVATE "${OFX_SDK_DIR}/include")

# Name the file MyFirstPlugin.ofx instead of libMyFirstPlugin.dll.
set_target_properties(MyFirstPlugin PROPERTIES PREFIX "" SUFFIX ".ofx")

# Link the C++ runtime statically so the .ofx does not depend on VCRUNTIME140*.dll.
set_property(TARGET MyFirstPlugin PROPERTY
    MSVC_RUNTIME_LIBRARY "MultiThreaded$<$<CONFIG:Debug>:Debug>")
```

Why the static runtime? A working third-party plugin on the test machine depended only on a few system DLLs, so the plugin was switched to the static runtime to match it. After that it depended only on KERNEL32.dll. Whether the runtime setting was needed to make Resolve load the plugin was **not isolated**. It is a safe default, because it removes one thing that could be missing on another PC.

# 6. Build, stage and install

The manual flow has three parts: build, copy into the bundle layout, copy to the OFX plugin folder. Doing it by hand each time is slow, so the course uses one batch file.

### What build-install.bat does, in order:

- Build with CMake in Release configuration.
- Copy the new MyFirstPlugin.ofx into `bundle\MyFirstPlugin.ofx.bundle\Contents\Win64`.
- Copy the whole bundle with xcopy to `C:\Program Files\Common Files\OFX\Plugins`.
- Delete log.txt so the next test starts with a clean log.

### Two rules, both learned the hard way:

- **Run it as administrator.** Without it, the copy to Program Files fails. In VS Code that means starting VS Code itself as administrator.
- **Close Resolve completely first.** A loaded .ofx is locked by the host, so it cannot be overwritten while Resolve runs.

After a build, confirm the plugin file itself is healthy (run in an x64 Developer Command Prompt, **Tested**):

```
dumpbin /exports MyFirstPlugin.ofx
```

You should see `OfxGetPlugin` and `OfxGetNumberOfPlugins` with plain names (no leading underscore, no @ and number). If they are missing or decorated, Resolve cannot find the entry points.

```
dumpbin /dependents MyFirstPlugin.ofx
```

With the static runtime this showed only KERNEL32.dll.

# 7. Making the plugin appear in Resolve

Open Resolve after installing. The route that worked (**Tested, Resolve-specific**):

- Resolve scans the OFX plugin folder at start-up. Restart it after every install.
- Check **Preferences, Video Plugins**. The bundle can be listed there as **Failed to load** and unticked. Re-enabling it was needed once on the test machine.
- On the **Edit** page, open Effects, then OpenFX. The plugin appears under the **grouping** you set in Describe (here: OFX Learning, then My First Plugin).
- Apply it to a clip, then select the clip and look in the Inspector for the plugin settings.

> **Not tested:** the Fusion page and the Color page. Everything above was done on the Edit page. OFX plugins are generally available in more than one page of Resolve, but check it yourself instead of trusting this guide, and tell your readers what you saw.

### A useful control experiment

If you are not sure whether your *setup* or your *plugin* is the problem, find any third-party OFX plugin that already shows up in Resolve and compare. On the test machine that comparison (what DLLs it depends on, where it is installed, where it appears in the menu) found the runtime difference quickly.

# 8. Troubleshooting: symptom, cause, fix

These are the real symptoms from the learning route, in the order they appeared.

| Symptom | Layer | What it meant and what fixed it |
|---|---|---|
| Plugin not in the Effects list at all | Resolve integration | Preferences, Video Plugins showed the bundle as **Failed to load** and unticked. Enabling it again made it appear. Also verify the bundle path and the exports (section 6). |
| Plugin shows in the list, but applying it says "No parameter is exposed to user or OFX Plugin [id] is not available" | Plugin registration and lifecycle | The log showed Resolve creating and destroying the instance about 120 times and never calling Render. The plugin was returning `kOfxStatReplyDefault` for CreateInstance and DestroyInstance. The fix was returning `kOfxStatOK` for both. **Caveat:** the pixel depths (Float, Short, Byte) were declared at the same time, so the exact decisive change was not isolated. Do both. |
| Plugin applied, but the picture does not change | Rendering | Render returned ReplyDefault, which does nothing. A filter must write the output image itself. |
| Build says it cannot copy / access denied | Install | Not running as administrator, or Resolve still open and locking the .ofx. |
| Resolve shows a "Use Alpha" checkbox though you defined no parameters | Resolve-specific | **Unexplained** on the test machine. Probably something the host adds itself, but this is a hypothesis. Not verified in the headers. |

# 9. The debug log trick

You cannot attach a debugger easily to a plugin inside Resolve, so write one line to a text file for every action. This is what showed that Render was never being called. The log is the single most useful tool in this guide.

```
static void logLine(const char *text)
```

```
{
    FILE *f = nullptr;
    if (fopen_s(&f, "D:\\DavinciCoding\\OpenfxProject\\MyFirstPlugin\\log.txt", "a") == 0 && f) {
        fprintf(f, "%s\n", text);
        fclose(f);
    }
}
// In pluginMain, first line:   logLine(action);
```

Reading the log after a test tells you which actions Resolve really sends, in what order, and what it skips. A healthy run on the test machine looked like this: Load, Describe, DescribeInContext, CreateInstance, GetClipPreferences, BeginSequenceRender, GetRegionOfDefinition, GetRegionsOfInterest, IsIdentity, Render, EndSequenceRender. The plugin answers ReplyDefault to the ones it does not handle.

**Remove or disable the logging before sharing a plugin.** It writes to a hard-coded path on one machine.

## 10. Verify names in your own headers

Never write an OpenFX name or function signature from memory. Search your SDK. This habit prevented several build errors during the course. From any PowerShell or cmd window, using the full path so the current folder does not matter:

```
findstr /n /c:"kOfxParameterSuite" D:\DavinciCoding\OpenfxProject\openfx\include\*.h
```

The result is a file name, a line number and the matching line, for example the suite name defined in ofxParam.h. To see a whole function declaration, print only the lines you need (PowerShell):

```
Get-Content D:\...\openfx\include\ofxParam.h | Select-Object -Index (1054..1059)
```

Two cautions: a findstr hit often cuts off the end of a declaration, so print the lines around it. And the header comments contain important rules. One example from ofxParam.h: **paramGetValue must not be called from the render actions**, which must use `paramGetValueAtTime`.

## 11. Where to go next

Once the empty plugin loads, the course builds up in small steps:

- **Passthrough render:** fetch source and output images, copy rows using the row-stride value (it can be negative).
- **Invert:** the first visible effect. Float RGBA, invert R, G, B, keep alpha.
- **A parameter:** an Amount slider from 0 to 1. Declare it with paramDefine in DescribeInContext, read it with paramGetValueAtTime in Render, blend with: original + amount * (inverted - original).
- Then Brightness, Contrast, Tint, Saturation, Blur, masks and regions, performance, GPU.

### Known open questions at the time of writing

- Which of the two changes (pixel depth list, or CreateInstance returning OK) was the true fix for the "No parameter is exposed" message.
- Where the "Use Alpha" checkbox comes from.
- Whether the image is premultiplied or unpremultiplied alpha: not checked in the headers yet.
- Short and Byte pixel paths: declared, but Resolve always chose Float in the log, so they are untested.
- Behavior on the Fusion and Color pages.

**Three-line summary.** 1) A .ofx is a DLL with two exported functions and one entry function. 2) Install it as a .ofx.bundle with the Contents\Win64 layout into the Common Files\OFX\Plugins folder, as administrator, with Resolve closed. 3) Log every action, and never return ReplyDefault for CreateInstance/DestroyInstance or Render if you want something to happen.